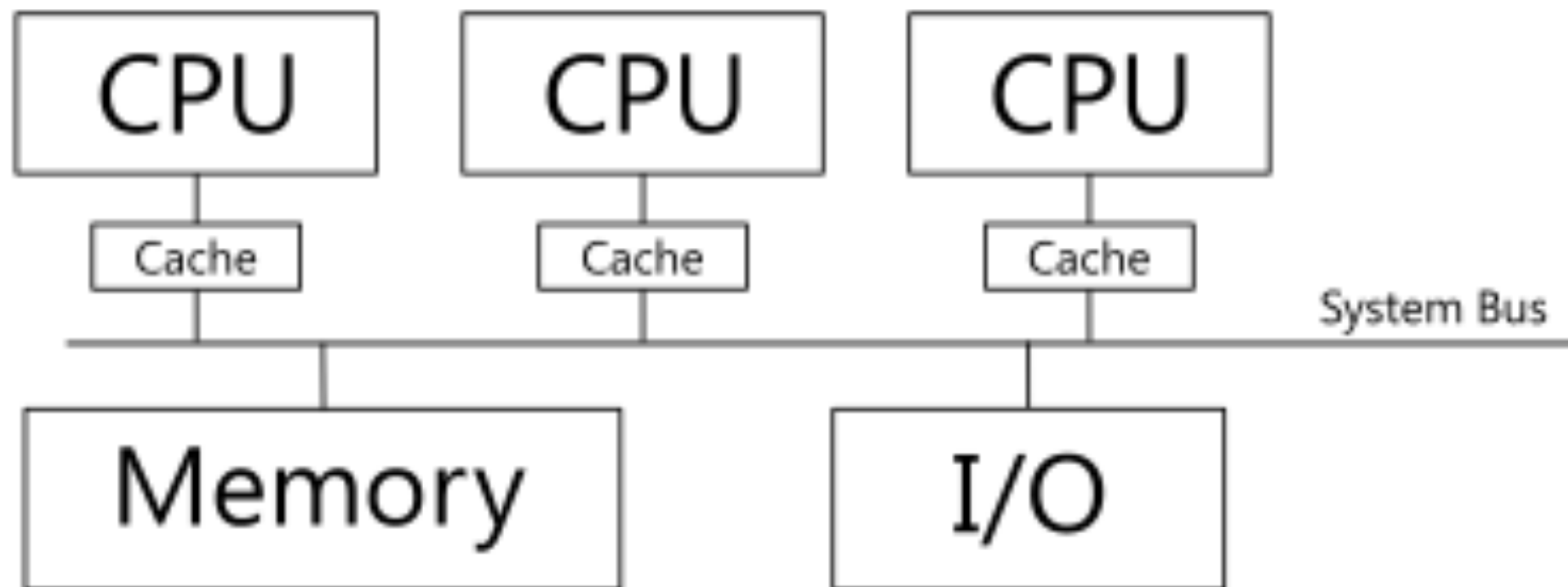


MIMD с разделяемой памятью.

Все процессоры совместно обращаются к общей памяти, обычно, через шину или иерархию шин. На практике масштабируемость этой архитектуры обычно приводит к некоторой форме иерархии памяти. Частота обращений к общей памяти может быть уменьшена за счет сохранения копий часто используемых данных в кэш-памяти, связанной с каждым процессором. Доступ к этому кэш-памяти намного быстрее, чем непосредственно доступ к общей памяти.



Современный многопроцессорный MIMD позволяет работать над несколькими задачами в параллельном режиме. На практике чаще всего программисты имеют дело с некоторой абстракцией, именуемой ядром операционной системы, которая берет на себя планирование выполнения процессов на том или ином ядре в тот или иной момент времени. В процесс входит не только код программы, а так же ресурсы — память, дескрипторы файлов, объекты IPC, очереди сигналов.

В классических UNIX системах единицей выполнения был процесс, и не было встроенной поддержки потоков в современном понимании. По сути процесс это изолированный экземпляр программы со своим адресным пространством, таблицей файловых дескрипторов и тд. UNIX разрабатывался для выполнения независимых задач, где изоляция процессов считалась преимуществом. Многозадачность достигала путем формирования пула процессов через вызовы `fork`, `wait`, `exec`. Процессы, не обладая единым адресным пространством, могли обмениваться информацией между собой с помощью анонимных каналов (`pipes`), именованных каналов (`fifo`), сигналов, разделяемой памяти, сокетов и тд.

Потоки были добавлены позже, чтобы решить проблемы, связанные с эффективностью. Переключение между потоками быстрее, чем переключение между процессами. Потоки используют общее адресное пространство, что позволяет не использовать надстройки вроде анонимных или именованных каналов и тд. Также в число преимуществ потоков входит время создания — обычно поток создаётся быстрее, чем процесс. Поэтому современные подходы в разработке программного обеспечения отдаёт предпочтение именно потокам, хотя не редко можно встретить поддерживаемые и сейчас программные комплексы основанные на процессах.

В Linux отличие между процессами и потоками выполнения менее явное, чем в некоторых других операционных системах. В Linux, процессы и потоки представлены в ядре с использованием единого объекта, называемого `struct task_struct`.

Процессы являются независимыми единицами выполнения, которые имеют свое собственное адресное пространство и ресурсы (файловые дескрипторы, счетчики и т. д.). В то время как потоки (или потоки выполнения) в Linux являются частными случаями процессов, которые разделяют некоторые или все свои ресурсы с родительским процессом, включая адресное пространство, файловые дескрипторы и другие ресурсы.

`task_struct` — это структура данных в ядре Linux, которая хранит информацию о процессе. Процесс — это инстанция выполняемой программы в операционной системе. В Linux процессы являются фундаментальными элементами для планирования и управления ресурсами. Структура `task_struct` содержит всю необходимую информацию для управления процессами и планирования задач. По сути все данные нитей также хранятся в такой же структуре.

```
struct task_struct {
    struct thread_info    thread_info; // флаги и биты состояния
    unsigned int          state;
    void                  *stack; // положение стека в пространстве ядра
    atomic_t              usage;
    unsigned int          cpu;
    int                   prio;
    struct                mm_struct *mm; // информация об управлении памятью
    int                   exit_state;
    int                   exit_code;
    int                   exit_signal;
    pid_t                 pid;          // id процесса
    pid_t                 tgid;         // id потока
    struct task_struct    *parent;
    struct list_head      children;
    u64                   start_time;
    const struct cred     *cred;        // учетные данные процесса
    struct signal_struct  *signal;      // Обработчики сигналов
    struct sighand_struct __rcu *sighand;
    struct files_struct   *files;       // указатель на структуру 'открытые файлы'
    struct thread_struct  thread;       // аппаратный контекст задачи
    char comm[TASK_COMM_LEN];          // имя задачи

```

...

Изначально любой процесс это всего лишь один поток. Однако к нему можно добавить и другие. Добавление может происходить например с помощью системного вызова

```
#include <sched.h>
int clone(int (*fn)(void *), void *stack, int flags,
void *arg);
```

`fn` — указатель на функцию, которая будет выполняться в новом потоке или процессе. Функция должна принимать один аргумент типа `void *` и возвращать целое число.

`child_stack` — указатель на начало стека, который будет использоваться новым потоком или процессом. Размер стека должен быть достаточным для выполнения функции и локальных переменных.

`flags` — целочисленное значение, определяющее, какие ресурсы будут разделяться между родительским и дочерним процессами или потоками. Флаги `CLONE_*` могут быть объединены с помощью оператора `|` (побитовое ИЛИ).

`arg`: указатель на аргумент, который будет передан функции `fn` в новом потоке или процессе.

Чтобы создать новый процесс с разделяемым адресным пространством, файловыми дескрипторами и другими ресурсами, вы можете использовать флаг `CLONE_VM` вместе с другими флагами, такими как `CLONE_FS`, `CLONE_FILES`, `CLONE_SIGHAND` и др.

Например:

```
int flags = CLONE_SIGHAND | CLONE_FS |  
CLONE_VM | CLONE_FILES;
```

```
int child_tid = clone(thread_function,  
child_stack, flags, &arg);
```


CLONE_PARENT — Если установлен флаг **CLONE_PARENT**, то родитель нового потомка будет таким же как и у вызывающего процесса.

CLONE_SIGHAND — флаг указывает, что дочерний процесс должен разделять таблицу обработчиков сигналов с родительским процессом. Это означает, что если изменяется обработчик сигнала в одном из процессов, эти изменения будут видны и в другом процессе.

CLONE_FS — Если флаг установлен, вызывающий процесс и процесс-потомок разделяют одну и ту же информацию о файловой системе. Она включает корневой каталог файловой системы, текущий рабочий каталог и значение `umask`. Любые вызовы `chroot`, `chdir`, или `umask` выполняемые вызывающим процессом или процессом-потомком также дают взаимный эффект. Если флаг **CLONE_FS** не установлен, процесс-потомок работает с копией информации по файловой системе от вызывающего процесса, снятой на момент вызова `clone`. Вызовы `chroot`, `chdir`, `umask` выполняемые позже одним из процессов не оказывают эффект на другой процесс.

CLONE_VM — Если флаг установлен, вызывающий процесс и процесс-потомок запускаются в том же пространстве памяти. В частности, записи в память, выполненные вызывающим процессом или процессом-потомком также видны из другого процесса. Кроме того, любые отражения памяти (`mapping`) или их завершение (`unmapping`) выполняемые через `mmap` или `munmap` потомком или вызывающим процессом, также оказывают эффект на другой процесс. Если флаг **CLONE_VM** не установлен, процесс-потомок запускается в отдельной копии пространства памяти вызывающего процесса, снятой на момент вызова `clone`. Записи в память или отражения выполняемые одним процессом не дают эффекта в другом процессе как и в `fork`.

CLONE_FILES — флаг указывает что, вызывающий процесс и процесс-потомок разделяют одну и ту же таблицу файловых дескрипторов. Если флаг **CLONE_FILES** не установлен, процесс-потомок наследует копии всех файловых дескрипторов, открытых в вызывающем процессе на время вызова `clone`.

CLONE_THREAD — Если флаг установлен, потомок размещается в той же группе тредов, что и вызывающий процесс. Если флаг **CLONE_THREAD** не установлен, то потомок размещается в своей собственной (новой) группе тредов, где ID группы такой же как и идентификатор процесса.

CLONE_VFORK — Если флаг установлен, то выполнение вызывающего процесса приостанавливается пока потомок не освободит свои ресурсы виртуальной памяти через вызов `execve` или `_exit` (как в `vfork`). Если флаг **CLONE_VFORK** не установлен, то после вызова и вызывающий процесс и процесс-потомок включаются в параллельную работу через системный планировщик и приложение не может знать в каком порядке будет осуществляться их выполнение.

CLONE_NEWNS — Запускает потомок в новом пространстве имён.

Каждый процесс живёт в некотором пространстве имён. Пространство имён процесса - это данные (список смонтированных файловых систем), описывающие файловую иерархию, видимую этим процессом. После вызова `fork` или `clone` где не установлен флаг **CLONE_NEWNS**, потомок живёт в том же пространстве имён, что и родитель. Системные вызовы `mount` и `umount` изменяют пространство имён вызывающего процесса и, с этого момента, оказывают эффект на все процессы, которые живут в этом же пространстве имён, но не оказывают эффект на процессы в других пространствах имён.

После вызова `clone` где флаг **CLONE_NEWNS** установлен, потомок запускается в новом пространстве имён, инициализированном копией пространства имён родителя.

Только привилегированный процесс может устанавливать флаг **CLONE_NEWNS**. Не допускается совместное использование флагов **CLONE_NEWNS** и **CLONE_FS** в одном вызове `clone`.

CLONE_PTRACE — Если флаг установлен и вызывающий процесс находится в режиме трассировки, то процесс-потомок также будет работать в режиме трассировки.

...

В случае успеха вызова `clone`, в вызывающий тред возвращается `PID` процесса-потомка. В случае ошибки, в контекст вызывающего процесса возвращается `-1`, процесс потомок не будет создан и значение `errno` устанавливается соответствующим образом.

Ошибки:

`EAGAIN` — уже запущено слишком много процессов.

`ENOMEM` — нет возможности выделить требуемую память для структуры, описывающей процесс-потомок или для копирования тех частей контекста вызывающего процесса, которые необходимо скопировать.

`EINVAL` — Возвращается `clone` когда для `child_stack` было задано нулевое значение.

`EINVAL` — Оба флага `CLONE_FS` и `CLONE_NEWNS` были заданы в аргументе `flags`.

`EINVAL` — `CLONE_THREAD` был задан, но `CLONE_SIGHAND` нет.

`EPERM` — `CLONE_PID` был задан процессом с ненулевым значением `PID`.

Дождаться завершения процесса потомка можно с помощью `waitpid`:

```
#include <sys/types.h>
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

В качестве параметра `pid` передается PID созданного процесса, `status` это out-параметр который вернет статус выполнения процесса. Значение `options` создается путем битовой операции ИЛИ над следующими константами:

`WNOHANG`: означает немедленный возврат, если ни один потомок не завершил выполнение.

`WUNTRACED`: также возвращаться, если есть остановленный потомок.

`WCONTINUED`: также возвращаться, если работа остановленного потомка возобновилась из-за получения сигнала `SIGCONT`.

Следующие значения `options`, присущие только Linux, используются для потомков, созданных с помощью `clone`:

`__WCLONE`: Ждать только клонированных потомков. Если не указано, то ожидаются только «не клонированные» потомки («клонированным» считается потомок, который не доставляет сигнал, или сигнал, отличный от `SIGCHLD`, своему родителю при завершении). Этот аргумент игнорируется, если также указано `__WALL`.

`__WALL` (начиная с Linux 2.4): Ждать всех потомков независимо от типа ("клонированный" или "неклонированный").

`__WNOTHREAD` (начиная с Linux 2.4): Не ждать потомков других нитей в той же группе нитей. Это поведение по умолчанию до Linux 2.4.

Пример создания и ожидания процесса с помощью clone.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sched.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

volatile int g_counter = 0;
#define STACK_SIZE 4096

int thread_func(void* ptr) {
    volatile int* shouldTerminate = ptr;
    while (!*shouldTerminate) {
        printf("[thread] = %d\n", g_counter); sleep(1);
    }
    printf("[thread] Terminating...\n");
    return 0;
}

int main() {
    printf("My pid: %d\n", getpid());
    void* stack = aligned_alloc(4096, STACK_SIZE);
    int shouldTerminate = 0;
    pid_t tid = clone(thread_func, stack + STACK_SIZE, CLONE_SIGHAND | CLONE_FS | CLONE_VM | CLONE_FILES,
&shouldTerminate);
    if (tid == -1) {
        perror("clone()");
        return 1;
    }
    printf("Done! Thread pid: %d\n", tid);
    while (1) {
        if (getchar() == 'e') {shouldTerminate = 1; break; }
        g_counter++;
    }
    printf("Waiting for thread...\n");
    int status;
    if (waitpid(tid, &status, __WALL) == -1) {
        perror("waitpid()");
    }
    free(stack);
}
```

PID процесса можно посмотреть с помощью утилиты `ps`, в предыдущем примере можно увидеть сразу два процесса с названием полученного после компиляции бинарного файла с разными значениями PID.

В случае если включен флаг `CLONE_THREAD` то создается поток и дожидаться его помощью `waitpid` не получится.

Дождаться завершения потока можно через системный вызов `futex`. Посмотреть группу потоков процесса возможно также через утилиту `ps`:

```
ps -T -p <PID интересующего процесса>.
```

Пример создания и ожидания потока с помощью clone.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sched.h>
#include <stdlib.h>
#include <linux/futex.h>
#include <sys/syscall.h>

volatile int g_counter = 0;
#define STACK_SIZE 4096

volatile int thread_finished = 0;

int thread_func(void* ptr) {
    volatile int* shouldTerminate = ptr;
    while (!*shouldTerminate) {
        printf("[thread] = %d\n", g_counter); sleep(1);
    }

    printf("[thread] Terminating...\n");

    __atomic_store_n(&thread_finished, 1, __ATOMIC_SEQ_CST);
    syscall(SYS_futex, &thread_finished, FUTEX_WAKE, 1, NULL, NULL, 0);

    return 0;
}

int main() {
    printf("My pid: %d\n", getpid());
    void* stack = aligned_alloc(4096, STACK_SIZE);
    int shouldTerminate = 0;
    pid_t tid = clone(thread_func, stack + STACK_SIZE, CLONE_THREAD | CLONE_SIGHAND | CLONE_FS | CLONE_VM | CLONE_FILES,
&shouldTerminate);
    if (tid == -1) {
        perror("clone()");
        return 1;
    }
    printf("Done! Thread pid: %d\n", tid);
    while (1) {
        if (getchar() == 'e') {shouldTerminate = 1; break; }
        g_counter++;
    }
    printf("Waiting for thread...\n");

    while (__atomic_load_n(&thread_finished, __ATOMIC_SEQ_CST) == 0) {
        syscall(SYS_futex, &thread_finished, FUTEX_WAIT, 0, NULL, NULL, 0);
    }

    free(stack);
}
```


POSIX Threads.

POSIX (Portable Operating Systems Interface for unⁱX — переносимый интерфейс операционных систем для UNIX-подобных систем) — это семейство стандартов, разработанных IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости между различными операционными системами. Стандарт POSIX определяет набор интерфейсов и функций, которые должны быть доступны в операционной системе, чтобы она считалась совместимой с POSIX.

- Набор рекомендаций POSIX Threads состоит из около 100 функции.
- Названия функций начинаются с `pthread_`. Функции можно разделить на 4 группы: управление потоками, мьютексы, условные переменные, синхронизация.
- Для использования необходимо подключить заголовочный файл `<pthread.h>`, указать `-lpthread` во время сборки.
- Потоки `pthread` используют `fork-join` модель, то есть потоки могут порождать другие потоки и ждать их завершения.
- Выполнение потоков планируется ОС, они могут выполняться последовательно или параллельно, могут назначаться на произвольные ядра.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4

typedef struct thread_data_s {
    int tid;
    float stuff;
} thread_data_t;

void* thr_func(void* arg) {
    thread_data_t *data = (thread_data_t *)arg;
    printf("Hello from thr_func, thread id: %d\n", data->tid);
    pthread_exit(NULL);
}

int main(int argc, char **argv) {
    pthread_t thr[NUM_THREADS]; int i, rc;

    thread_data_t thr_data[NUM_THREADS];

    for(i = 0; i< NUM_THREADS; ++i) {
        thr_data[i].tid = i;
        if((rc = pthread_create(&thr[i], NULL, thr_func, &thr_data[i]))) {
            fprintf(stderr, "error: pthread_create, rc: %d\n", rc);
            return EXIT_FAILURE;
        }
    }

    for(i = 0; i< NUM_THREADS; ++i) {
        pthread_join(thr[i], NULL);
    }
    return EXIT_SUCCESS;
}
```

Создание нового потока начинается с вызова функции:

```
int pthread_create(pthread_t *thr, const  
pthread_attr_t *attr, void* (*start_routine)  
(void*), void *arg)
```

Функция `pthread_create` возвращает 0 если в случае успешного вызова или код ошибки.

Некоторые коды ошибок:

EAGAIN – у системы нет ресурсов для создания нового потока, или система не может больше создавать потоков, так как количество потоков превысило значение **PTHREAD_THREADS_MAX**.

EINVAL – неправильные атрибуты поток.

EPERM – Вызывающий поток не имеет должных прав для того, чтобы задать нужные параметры или политики планировщика.

```
int pthread_create(pthread_t * thread,  
const pthread_attr_t *attr, void  
*(*start_routine) (void *)  
start_routine, void *arg);
```

Аргументы для `pthread_create`:

`thread` — идентификатор потока (для завершения, синхронизации и т.д.).

`attr` — параметры потока (например, минимальный размер стека).

`start_routine` - функция для запуска (принимает и возвращает указатель `void*`).

`arg` — аргументы функции.

Атрибуты задают свойства потока. Если вместо атрибутов передать NULL, то атрибуты будут заданы по умолчанию. Объект задающий атрибуты потока имеет тип `pthread_attr_t`. Такой объект должен быть инициализирован с помощью функции

```
int pthread_attr_init(pthread_attr_t *attr);
```

В результате объект будет содержать набор свойств потока по умолчанию для данной реализации потоков. А ресурсы, которые могут использоваться в системе для хранения этих атрибутов освобождаются вызовом функции (после того, как объект был использован в вызове `pthread_create` и больше не нужен)

```
int pthread_attr_destroy(pthread_attr_t *attr);
```

Поток может быть "присоединяемым" (`joinable`) или "оторванным" (`detached`). Для установки этого свойства в атрибутах используется функция

```
int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate);
```

, где `detachstate` можно установить в `PTHREAD_CREATE_JOINABLE` или в `PTHREAD_CREATE_DETACHED` соответственно.

По умолчанию создается атрибут с параметрами:

Атрибут `scope`: `PTHREAD_SCOPE_PROCESS`;

Новый поток не присоединен ни к одному процессу.

Атрибут `detachstate`: `PTHREAD_CREATE_JOINABLE`;

Нити создаются присоединенными (для освобождения ресурсов после завершения нити необходимо вызвать `pthread_join`).

Атрибут `stackaddr`: `NULL`; Новый поток получает адрес стека, выделенного системой.

Атрибут `stacksize`: `0`; Новый поток имеет размер стека, определенный системой.

Атрибут `priority`: `0`; Имеет приоритет `0`.

Атрибут `inheritsched`: `PTHREAD_EXPLICIT_SCHED`;

Нить не наследует приоритет родительского потока.

Атрибут `schedpolicy`: `SCHED_OTHER`;

Нить использует фиксированные приоритеты, задаваемые ОС.

Используется вытесняющая многозадачность (нить исполняется, пока не будет вытеснена другой нитью или не заблокируется на примитиве синхронизации)

Для того, чтобы освободить память выделенную для атрибутов во время инициализации необходимо вызвать функцию `pthread_attr_destroy`:

```
int pthread_attr_destroy(pthread_attr_t  
*attr);
```

После успешного завершения возвращает 0 или любое другое значение в случае ошибки.

Для каждого присоединяемого потока, один из других потоков явно должен вызвать функцию:

```
int pthread_join(pthread_t thread, void  
**value_ptr);
```

Процедура `pthread_join` блокирующая и поток, вызвавший эту функцию, останавливается, пока не окончится выполнение потока `thread`. Если никто не вызывает `pthread_join` для присоединяемого потока, то завершившись, он не освобождает свои ресурсы, а это может служить причиной утечек памяти в программе.

Аргумент `value_ptr` это указатель на указатель, возвращенный функцией завершившегося потока.

Вызов функции `pthread_join( )` для идентификатора уже присоединенного потока может привести к непредсказуемым последствиям; например, мы можем присоединиться к потоку, который был создан позже и повторно использует тот же идентификатор.

Потоки не имеют иерархии. Любой поток в процессе может воспользоваться функцией `pthread_join( )` для присоединения к другому потоку в том же процессе.

К любому потоку можно присоединиться вызовом `pthread_join` и ожидать его завершения. Однако в некоторых случаях статус завершения потока и возврат значения не нужны. Необходимо завершить поток и автоматически выгрузить ресурсы обратно в распоряжение ОС. В таких случаях можно обозначить поток отсоединившимся используя вызов процедуры:

```
int pthread_detach(pthread_t thread);
```

При удачном завершении `pthread_detach ( )` возвращает код 0, ненулевое значение сигнализирует об ошибке.

Если поток отсоединён, его уже нельзя перехватить с помощью вызова `pthread_join ( )`, чтобы получить статус завершения. Также нельзя отменить его отсоединенное состояние.

Отсоединенный поток не становится устойчивым к вызову `exit( )`, сделанному в другом потоке, или к инструкции `return`, выполненной в главной программе. В любой из этих ситуаций все потоки внутри процесса немедленно завершаются, вне зависимости от того, присоединяемые они или нет. Иными словами, функция `pthread_detach( )` просто отвечает за поведение потока после его завершения, но не за то, в каких обстоятельствах тот завершается.

`void pthread_exit (void *val_ptr)` – освобождает всю память, занятую данными нити, включая стек нити. Любые данные, помещенные в стек, становятся неопределенными, так как стек освобожден и соответствующая область памяти может использоваться другой нитью. Вызов `pthread_exit` делает объект, на который указывает параметр `val_ptr`, доступным другой нити исполнения, например, в породившей завершившуюся нить.

```

#define DETACHED
void* thr_func(void* arg) {
#ifdef DETACHED
    for(int i = 0 ; i < 3 ; ++i){
        usleep(300000); puts("Hello from detached thread");
    }
#else
    for(int i = 0 ; i < 3 ; ++i){
        usleep(300000); puts("Hello from joined thread");
    }
#endif
    pthread_exit(NULL);
}
int main(int argc, char **argv) {
    int rc; pthread_t thr;
    if((rc = pthread_create(&thr, NULL, thr_func, NULL))) {
        fprintf(stderr, "error: pthread_create, rc: %d\n", rc);
        return EXIT_FAILURE;
    }
#ifdef DETACHED
    pthread_detach(thr);
    for(int i = 0 ; i < 3 ; ++i){
        puts("Some work in main thread...");
        usleep(500000);
    }
#else
    pthread_join(thr, NULL);
    for(int i = 0 ; i < 3 ; ++i){
        puts("Write this after join");
        puts("Some work...");
        usleep(500000);
    }
#endif
    return EXIT_SUCCESS;
}

```

Для отмены выполнения нити существует функция `pthread_cancel`:

```
int pthread_cancel (pthread_t thread_id);
```

При удачном завершении возвращает 0, при ошибочном положительные значения. Выполняет запрос на отмену потока по его `thread_id`. Может быть выполнен немедленно, проигнорирован или выполнен чуть позже. Для определения дальнейшего поведения потока есть две функции:

```
int pthread_setcancelstate(int state, int *oldstate);
```

Вызывается, чтобы проигнорировать или принять запрос на завершение. Запрос игнорируется, если аргумент `state` является `PTHREAD_CANCEL_DISABLE`, принимается если `state` имеет значение `PTHREAD_CANCEL_ENABLE`. Если принятие запроса разрешено то вызывается процедура:

```
int pthread_setcanceltype(int type, int *oldtype);
```

Если аргумент `type` имеет значение `PTHREAD_CANCEL_ASYNCHRONOUS` то остановка потока произойдет мгновенно. Если `type` имеет значение `PTHREAD_CANCEL_DEFERRED`, запрос на завершение откладывается до следующей точки завершения.

Точками завершения являются такие функции, где выполняется проверка на наличие ожидающего обработки запроса на завершение. Если такой запрос ждёт обработки, завершение выполняется сразу же. В общем случае любая функция, которая приостанавливает выполнение текущего потока в течение длительного времени, должна быть точкой завершения. Такие функции для работы с `pthread`, как `pthread_join`, `pthread_cond_wait`, `pthread_cond_timedwait` и `pthread_testcancel` тоже служат точками завершения.


```

int main() {
    int res;
    pthread_t a_thread;
    void *thread_result;
    res = pthread_create(&a_thread, NULL, thread_function, NULL);
    if (res != 0) { perror("Thread creation failed»); exit(EXIT_FAILURE);}
    sleep(3); printf("Canceling thread...\n");
    res = pthread_cancel(a_thread);
    if (res != 0) { perror("Thread cancelation failed»);
exit(EXIT_FAILURE);}
    printf("Waiting for thread to finish...\n");
    res = pthread_join(a_thread, &thread_result);
    if (res != 0) { perror("Thread join failed"); exit(EXIT_FAILURE);}
    exit(EXIT_SUCCESS);
}

void *thread_function(void *arg) {
    int i, res;
    res = pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    if (res != 0) {
        perror("pthread_setcancelstate failed");exit(EXIT_FAILURE);
    }
    res = pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);
    if (res != 0) {
        perror{"Thread pthread_setcanceltype failed");
        exit(EXIT_FAILURE);
    }
    printf("thread_function is runningn");
    for(i = 0; i < 10; i++) { printf("still run (%d)\n», i);sleep(1);}
    pthread_exit(0);
}

```


Каждый поток внутри процесса имеет свой уникальный идентификатор. Он возвращается в вызывающий поток функцией `pthread_create()`.

```
pthread_t pthread_self(void);
```

Кроме того, с помощью функции `pthread_self()` поток может получить собственный идентификатор.

В традиционном программном интерфейсе UNIX переменная `errno` является глобальной и целочисленной. Однако этого может привести к ошибке для многопоточных программ. Если поток вызвал функцию, записавшую ошибку в глобальную переменную `errno`, это может ввести в заблуждение другие потоки, которые тоже вызывают функции и проверяют значение `errno`. Иными словами, результатом будет состояние гонки. Таким образом, в многопоточных программах каждый поток имеет свой отдельный экземпляр `errno`.

В `Linux` (и в большинстве реализаций `UNIX`) для этого используется примерно один подход: `errno` объявляется в виде макроса, который разворачивается в вызов функции, возвращающий изменяемое значение вида `lvalue`, уникальное для каждого отдельного потока.

Механизм `errno` был интегрирован в потоки так, что процедура оповещения об ошибках полностью соответствует традиционному подходу, применяющемуся в программных интерфейсах `UNIX`.

C11 Threads.

В стандарте языка C11 официально появилась многопоточность. Создать поток можно с помощью функции `thrd_create`, предварительно подключив заголовочный файл `<threads.h>`.

```
int thrd_create(thrd_t *thr, int (*func)
(void *), void *arg);
```

Вызов `thrd_create` принимает идентификатор потока `thr`, функцию потока `func`, аргументы потока `arg`, а возвращает код ошибки.

Ожидание завершения потока возможно с помощью функции `thrd_join`. Функция блокирует вызывающий поток, пока указанный поток не завершит выполнение. Принимает объект потока `thrd_t` и указатель на переменную для сохранения результата выполнения потока.

```
int thrd_join(thrd_t thr, int *res);
```

Отсоединение позволяет потоку продолжать работу независимо от вызывающего потока. Отсоединение возможно с помощью функции `thrd_detach`. Функция принимает объект потока `thrd_t`.

```
int thrd_detach(thrd_t thr);
```

Завершить поток можно с помощью функции `thrd_exit`:

```
void thrd_exit(int res);
```

Приостановка потока на некоторое время возможно с помощью функции `thrd_sleep`:

```
int thrd_sleep(const struct timespec  
*duration, struct timespec *remaining);
```

Функция останавливает текущий поток на длительность `duration`, помещая оставшееся до возобновления работы время в `remaining`.

Функция `thrd_yield` позволяет уступить свое время исполнения в пользу других потоков.

```
void thrd_yield(void);
```

Функция `thrd_current` позволяет получить объект текущего потока:

```
thrd_t thrd_current(void);
```

Функция сравнения двух объектов потоков `thrd_t`. Возвращает ненулевое значение, если они равны, и ноль в противном случае.

```
int thrd_equal(thrd_t lhs, thrd_t rhs);
```